PROBLÈME · ANNALES MP · 1 H 15

Flots de données paresseux

OCaml exclusivement. `let list_finite = [1; 2; 3]` et `let rec list_ones = 1 :: list_ones`.

1. Croquis en maillons de `list_finite` et `list_ones` en mémoire ; quelle région de la mémoire ?

Le code suivant provoque `Stack overflow during evaluation (looping recursion?)` :

```
let list_fibo : int list =
  let rec list_fibo_with_internal_state (a:int) (b:int) : int list =
    a :: (list_fibo_with_internal_state b (a+b))
  in
  list_fibo_with_internal_state 0 1
```

1. Traduire *stack* ; expliquer l'erreur.
2. Avec `ocamlopt fibo.ml -o fibo` puis `./fibo` : l'une des étapes échoue-t-elle ?

`type 'a stream = Nil | Cons of 'a * (unit -> 'a stream)`. Exemples : `let rec ones = Cons (1, fun () -> ones)` ; `fibo` défini par `fibo_with_internal_state a b = Cons (a, fun () -> fibo_with_internal_state b (a+b))`.

1. Constante `integers : int stream` (entiers naturels croissants).
2. `range (a:int) (b:int) : int stream` (entiers de a inclus à b exclu).
3. Toute suite (σ_n) de chaînes est-elle codée par un `string stream` ?
4. `hd (u:'a stream) : 'a`, avec `Failure "hd"` si `u` est vide.
5. `tl (u:'a stream) : 'a stream`, avec `Failure "tl"`.
6. `of_list (l:'a list) : 'a stream`.
7. `iter (f:'a -> unit) (t:int) (u:'a stream) : unit` : applique `f` aux `t` premières valeurs existantes.
8. `map (f:'a -> 'b) (u:'a stream) : 'b stream`.
9. `zip (w:'a stream array) : 'a array stream` : flot des tableaux $[u_n, v_n, \dots]$, vide dès que l'un des flots l'est.
10. `intertwin (u:'a stream) (v:'a stream) : 'a stream` : alterne les éléments de `u` et `v`, en continuant avec l'autre quand l'un est épuisé.

```
let rec product_wrong u1 u2 =
  match u1, u2 with
  | _, Nil | Nil, _ -> Nil
  | Cons (hd1, tl_begetter1), Cons (hd2, tl_begetter2) ->
      let part1 = product_wrong (tl_begetter1 ()) u2 in
      let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
      Cons ((hd1, hd2), fun () -> intertwin part1 part2)
```

1. Montrer que `product_wrong u1 u2` ne termine pas toujours, et qu'il suffit de déplacer quelques mots pour le corriger.
2. Signification du `0` dans `return 0` (C) ou `exit 0` (OCaml).
3. Énoncer le théorème de Turing sur l'arrêt.
4. On admet une fonction `run : string -> int -> int` qui termine toujours et renvoie 0 si le code x termine sans erreur en moins de n étapes, 1 sinon. Construire ou réfuter l'existence de `filter (f:'a -> bool) (u:'a stream) : 'a stream` (éléments de `u` vérifiant `f`, dans l'ordre ; par exemple `filter is_negative ones` vaut `Nil`).